

Aula 4 — PWA Offline-First

"Instale uma vez, funcione em qualquer lugar — mesmo sem internet"

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 01/07/2026

Recap da Aula 3

- Flutter vs React Native: isolates x event loop, Impeller, Pixel Perfect, Liquid Glass
- Dart: tipos, null safety, Future/async, widgets Stateless vs Stateful, setState
- Apps em produção Flutter: Nubank, Alibaba, BMW, Google Play, Toyota
- Atividade 3 (native module + comparativo) — bastante gente entregou 

Alguém ainda não entregou? Bot ainda aceita PR tardio.

Agenda da aula (3h30)

Bloco 1 (75min) — PWA: conceito → demo ao vivo → teoria → mão na massa

→ O que é PWA · Demo Flutter Shell · SW · Manifest · Cache · Setup A4 + TODO 1

Intervalo (30min)

Bloco 2 (75min) — Offline-first em profundidade + contexto

→ Workbox · IndexedDB · vite.config.ts · Lighthouse demo · RN-Web · Capacitor · Enunciado A4

| KMP movido para **Aula 5 (08/07)** junto com BFF/GraphQL.

Objetivos de aprendizagem

PWA:

- Compreender ciclo de vida de Service Workers (install, activate, fetch)
- Aplicar cache strategies com Workbox (NetworkFirst, CacheFirst)
- Converter SPA em app instalável e offline-first com Lighthouse CI

Hybrid shell (bônus):

- Entender como PWA roda dentro de shell Flutter/RN via WebView
- Detectar contexto (UA `FlutterShell`) e adaptar comportamento

"PWA é só web?" — fato vs ficção

Twitter Lite (*Eng Blog, 2017*): offline real, 75% mais tweets enviados.

Pinterest (*web.dev, 2017*): 60% mais engajamento, instalável.

Spotify Web (2023): PWA instalável no desktop — mesmo conteúdo do app.

Forbes (2016): 100ms de melhoria no TTI → +1% de sessões.

PWA não é cosmético. Mas não substitui nativo em todos os casos — limitações reais no iOS existem (sem push até iOS 16.4, sem Background Sync). Próximos slides detalham.

⚠️ Dados acima: blogs de engenharia das próprias empresas (não estudos controlados independentes).

Demo ao vivo — Flutter Shell → PWA

```
cd exercicios/04-pwa-tmdb/demo/flutter-pwa-bridge  
flutter pub get  
flutter run
```

O que observar:

1. Splash Flutter nativo (animação Dart) → 2s
2. Fade para o WebView (transição suave)
3. PWA carrega com tab bar, cards, favoritos
4. ✈️ Modo avião → PWA ainda funciona (Service Worker + cache!)
5. DevTools → Application → Service Workers: registrado dentro do WebView

Ponto pedagógico: o Service Worker registra e faz cache mesmo dentro de um WebView Flutter — offline funciona de graça.

Anatomia de uma PWA



HTTPS

Origem segura obrigatória — SW só registra em HTTPS (ou localhost)



Service Worker

Proxy de rede — intercepta fetch, gerencia cache, habilita offline



Web App Manifest

Metadados: nome, ícones, cores,
`display: standalone`

Precisa dos 3 para ser instalável. HTTPS + SW = offline. Manifest = "cara de app nativo".

Por que o PWA funciona offline?

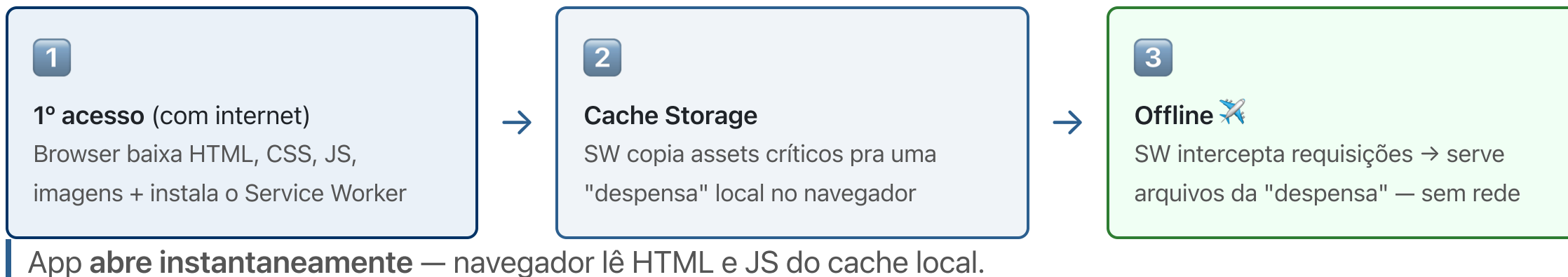
"Para exibir qualquer coisa, o navegador precisa de HTML, CSS e JS.
A mágica do PWA não é fazer esses arquivos aparecerem do nada —
é como ele os armazena no seu dispositivo."

O segredo tem nome: **Service Worker**



Decide em runtime: *"pego da internet ou sirvo do cache?"*

Como o PWA funciona offline — passo a passo



E os dados dinâmicos? — IndexedDB

Cache Storage (Service Worker)

HTML · CSS · JS · imagens — o *esqueleto* do app

← instalação / 1ª visita

+

IndexedDB (idb)

Filmes · favoritos · posts — o *conteúdo* do usuário

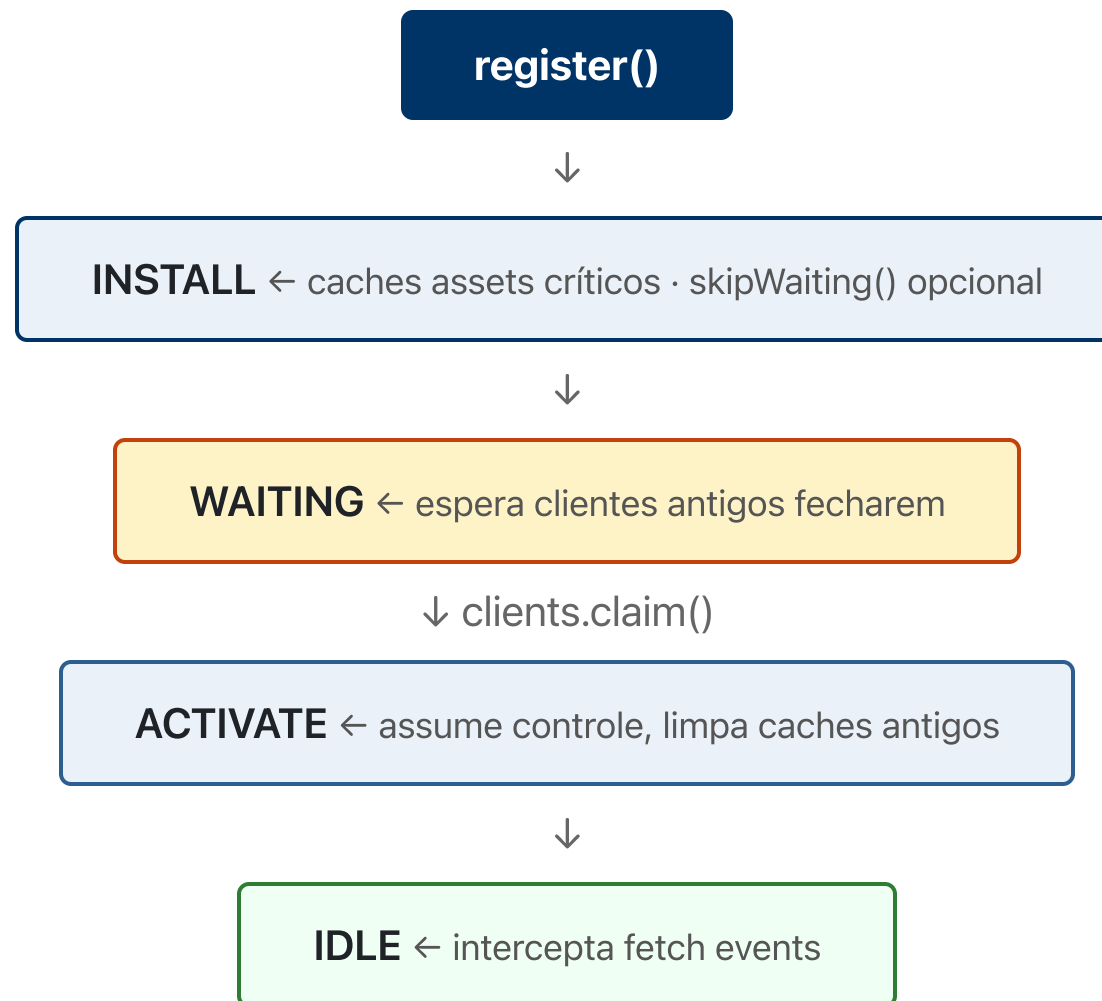
← ao usar o app online

No A4: SW serve o app → React lê filmes do IndexedDB → funciona offline sem precisar de rede. ✓

Web App Manifest

```
{
  "name": "Minha PWA",
  "short_name": "MinhaApp",
  "description": "Descrição curta da app",
  "start_url": "/",
  "scope": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#003366",
  "icons": [
    {
      "src": "/icons/icon-192.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/icons/icon-512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "any maskable"
    }
  ]
}
```

Service Worker — lifecycle



Service Worker — install + activate

```
// sw.js
const CACHE_NAME = 'app-v3';
const PRECACHE_URLS = ['/', '/index.html', '/styles.css', '/app.js'];

self.addEventListener('install', (event) => {
  event.waitUntil(
    caches.open(CACHE_NAME).then((cache) => cache.addAll(PRECACHE_URLS))
  );
  self.skipWaiting(); // assume controle imediato (cuidado!)
});

self.addEventListener('activate', (event) => {
  event.waitUntil(
    caches.keys().then((names) =>
      Promise.all(
        names.filter((n) => n !== CACHE_NAME).map((n) => caches.delete(n))
      )
    )
  );
  self.clients.claim();
});
```

Service Worker — anatomia do `fetch` event

```
self.addEventListener('fetch', (event) => { // ①
  if (event.request.url.includes('/static/')) { // ②
    event.respondWith( // ③
      caches.match(event.request).then((cached) => { // ④
        return cached || fetch(event.request).then((res) => { // ⑤
          caches.open(CACHE).then(c => c.put(event.request, res.clone())); // ⑥
          return res;
        });
      });
    });
  });
});
```

① `self.addEventListener('fetch')`

Dispara em *toda* req do app — HTML, CSS, JS, API. SW age como proxy.

③ `event.respondWith()`

"Sequestra" o fluxo. Navegador espera você entregar a resposta — cache ou rede.

⑤ `cached || fetch()`

Cache hit → serve imediato (offline ✓). Cache miss → vai à rede e cacheia pra próxima.

② `if ('/static/')`

Roteador: estratégia diferente por tipo de URL (Cache-first p/ assets, Network-first p/ API).

④ `caches.match()`

Vasculha Cache Storage. Retorna arquivo salvo ou `undefined` se não cacheado ainda.

⑥ `res.clone()`

Body é stream único — lê só uma vez. Clone vai pro cache; original vai pro navegador.

Cache strategies

Cache-first (estáticos)

Tenta cache; se falhar, vai pra rede; cacheia o resultado.

Network-first (dados dinâmicos)

Tenta rede com timeout; se falhar, vai pro cache.

Stale-While-Revalidate

Serve cache imediatamente (rápido) + busca rede em paralelo (atualiza próxima request).

Cache-only / Network-only

Casos específicos (assets imutáveis vs telemetria).

Cache-first — implementação

```
self.addEventListener('fetch', (event) => {  
  if (event.request.url.includes('/static/')) {  
    event.respondWith(  
      caches.match(event.request).then((cached) => {  
        return cached || fetch(event.request).then((response) => {  
          const clone = response.clone();  
          caches.open(CACHE_NAME).then((cache) => cache.put(event.request, clone));  
          return response;  
        });  
      });  
    );  
  }  
});
```

Use para JS, CSS, imagens, fonts versionadas.

Stale-While-Revalidate — implementação

```
self.addEventListener('fetch', (event) => {  
  if (event.request.url.includes('/api/feed')) {  
    event.respondWith(  
      caches.open(CACHE_NAME).then(async (cache) => {  
        const cached = await cache.match(event.request);  
        const fetchPromise = fetch(event.request).then((response) => {  
          cache.put(event.request, response.clone());  
          return response;  
        });  
        return cached || fetchPromise;  
      })  
    );  
  }  
});
```

Usuário vê algo imediatamente, próxima carga já está atualizada.

A4 — Atualize seu fork antes de começar

```
# 1. Adicionar upstream (só na primeira vez)
git remote add upstream https://github.com/jacksonsmith/puc-iec-mobile-multiplataforma.git

# 2. Buscar mudanças do prof
git fetch upstream

# 3. Mergear na sua branch main
git checkout main
git merge upstream/main

# 4. Agora crie sua branch de trabalho
git checkout -b feat/a4-pwa
```

Windows: mesmos comandos — Git Bash ou WSL.

A4 — Setup (3 passos)

```
# Passo 0 — confirmar que está na pasta certa
cd exercicios/04-pwa-tmdb/pratica
ls src/services           # deve listar: tmdb.ts db.ts
```

Windows: `cd exercicios\04-pwa-tmdb\pratica` e `dir src\services`

```
# Passo 1 — instalar dependências
npm install

# Passo 2 — configurar token TMDB
cp .env.example .env.local      # Windows cmd: copy .env.example .env.local
# edite .env.local → VITE_TMDB_TOKEN=<seu-token-aqui>
# Gere em: themoviedb.org/settings/api → API Read Access Token

# Passo 3 — rodar
npm run dev                     # dev server: http://localhost:5173
```

A4 — O que implementar (TODO 1)

Arquivo: `src/services/tmdb.ts`

```
// STUB atual - faz os testes falharem ↓
export async function fetchPopularMovies(_page = 1): Promise<MoviesResponse> {
  throw new Error('TODO 1: fetchPopularMovies não implementada');
}

// Sua implementação - substitua o throw ↓
export async function fetchPopularMovies(page = 1): Promise<MoviesResponse> {
  const { data } = await tmdbClient.get<MoviesResponse>('/movie/popular', {
    params: { language: 'pt-BR', page },
  });
  return data;
}
```

`tmdbClient` é um axios já configurado com `baseUrl` e `Authorization: Bearer` no arquivo.

Intervalo — 30min

Volta às :____

Aproveita pra:

- Esticar perna
- Pegar token TMDB se não tiver: developer.themoviedb.org → API → Bearer
- Abrir Chrome DevTools → aba **Application**

Próximo bloco: PWA com Service Workers. Chrome DevTools aberto na aba **Application**.

PWA — por que os dados não vêm só do Service Worker?

Duas camadas de cache neste app:

Camada	O que cacheia	API
Workbox (SW)	Respostas HTTP completas (headers + body)	CacheStorage
IndexedDB (idb)	Objetos JS tratados (só results[])	indexedDB

Por que as duas?

- SW cache = resposta HTTP bruta → bom pra filmes estáticos, ruim pra paginar
- IndexedDB = dados por page-N → controle fino, offline sem SW
- Juntos: SW intercepta requests HTTP, IndexedDB guarda estado da lista

Este app usa **as mesmas estratégias do slide** (NetworkFirst/CacheFirst/SWR) mas na camada de dados.

O que é o Workbox?

Workbox é uma biblioteca do Google que abstrai a API de Service Worker.

Sem Workbox, você escreve cache manual para cada rota:

```
self.addEventListener('fetch', event => {  
  event.respondWith(  
    caches.match(event.request).then(cached => cached || fetch(event.request))  
  );  
});
```

Com Workbox, você declara a estratégia:

```
registerRoute(/\.png$/, new CacheFirst());
```

Sem Workbox	Com Workbox
~150 linhas de boilerplate	~10 linhas declarativas

Workbox é para Service Workers o que o Axios é para `fetch` — você podia fazer sem, mas não vale a pena.

Workbox — abstração madura

```
import { precacheAndRoute } from 'workbox-precaching';
import { registerRoute } from 'workbox-routing';
import {
  CacheFirst,
  NetworkFirst,
  StaleWhileRevalidate,
} from 'workbox-strategies';

precacheAndRoute(self.__WB_MANIFEST);

registerRoute(/\.(?:png|jpg|svg)$/ , new CacheFirst({ cacheName: 'images' }));
registerRoute(
  /\api\/static\/ ,
  new StaleWhileRevalidate({ cacheName: 'api-static' })
);
registerRoute(/\api\/dynamic\/ , new NetworkFirst({ cacheName: 'api-dynamic' }));
```

Workbox elimina boilerplate. Use em produção.

IndexedDB — persistência local estruturada

`localStorage` é síncrono e limitado (~5MB). **IndexedDB** é a opção real.

```
import Dexie from 'dexie';

const db = new Dexie('MyAppDB');
db.version(1).stores({
  drafts: '++id, content, createdAt',
  user: '&id, name, email',
});

// Add
await db.drafts.add({ content: 'Hello', createdAt: Date.now() });

// Query
const recent = await db.drafts
  .orderBy('createdAt')
  .reverse()
  .limit(10)
  .toArray();
```

Dexie torna IndexedDB usável.

Background Sync API

Usuário fecha app sem conexão. Operação fica pendente. Quando conectividade volta, **Service Worker dispara sync event mesmo com app fechado.**

```
// Page side
async function submitOfflineSafe(message) {
  await db.outbox.add({ message, attempts: 0 });

  if ('serviceWorker' in navigator && 'SyncManager' in window) {
    const registration = await navigator.serviceWorker.ready;
    await registration.sync.register('flush-outbox');
  }
}

// sw.js
self.addEventListener('sync', (event) => {
  if (event.tag === 'flush-outbox') {
    event.waitUntil(flushOutbox());
  }
});
```

Web Worker vs Service Worker — quando usar cada um?

Web Worker — "preciso processar dados sem travar a UI"

```
// Parsing de CSV gigante sem congelar a página  
const worker = new Worker('parser.js');  
worker.postMessage(csvData);           // envia pra thread separada  
worker.onmessage = e => render(e.data); // recebe resultado
```

🎯 Casos: filtrar 50k registros, aplicar filtro em imagem, rodar modelo ML

Service Worker — "preciso interceptar rede e controlar cache"

```
// Responde a fetch do app inteiro, mesmo sem internet
self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request) || fetch(event.request)
  );
});
```

🎯 Casos: carregar app offline, cachear TMDB, push notification, sync em background

Web Worker = **thread paralela** pra sua página.

Service Worker = **proxy de rede** que persiste entre sessões.

Web Workers vs Service Workers

Aspecto	Web Worker	Service Worker
Propósito	Compute pesado off-thread	Network proxy + cache
Lifecycle	Scope da página	Persistente, instalado
Escopo	Página atual	Origin/scope inteiro
API	postMessage com page	fetch , cache , push , sync
Múltiplas instâncias	1 por chamada	1 por scope

Use Web Worker para parsing JSON gigante, image processing, ML em browser.

vite-plugin-pwa — como o Workbox é configurado

O plugin gera o Service Worker automaticamente a partir do `vite.config.ts`. Você **não escreve** `sw.js` manualmente.

```
// vite.config.ts (já pronto na pratica/)
VitePWA({
  registerType: 'autoUpdate',
  workbox: {
    globPatterns: ['**/*.{js,css,html,svg,png,woff2}'], // precache: app shell
    runtimeCaching: [
      {
        urlPattern: /^https:\/\/api\.themoviedb\.org\/$/,
        handler: 'NetworkFirst', // API: tenta rede → fallback cache
        options: { cacheName: 'tmdb-api', expiration: { maxEntries: 50, maxAgeSeconds: 86400 } },
      },
      {
        urlPattern: /^https:\/\/image\.themoviedb\.org\/$/,
        handler: 'CacheFirst', // Imagens: cache direto
        options: { cacheName: 'tmdb-images', expiration: { maxEntries: 200, maxAgeSeconds: 604800 } },
      },
    ],
  },
})
```

PWA — o que funciona (casos reais)

Todos os números abaixo são dados de engenharia publicados pelas próprias empresas — leia com olhos críticos: são cases de sucesso, não estudos controlados.

Pinterest (*web.dev case study, 2017*)

→ 60% mais engajamento · 44% mais receita de anúncios · 40% menos tempo de interação ↓

AliExpress (*web.dev case study, 2016*)

→ 104% mais conversão em novos usuários · 74% mais tempo de sessão no iOS

Twitter Lite (*Eng Blog, 2017 — arquivado*)

→ 65% mais páginas/sessão · 75% mais tweets enviados · bounce rate –20%

Padrão: são casos de *mobile web* melhorado (não nativo substituído). Ganhos são reais pra usuários que já estavam no browser — não comparam com native app direto.

PWA — o que não funciona (limitações reais)

iOS/Safari (até 2023):

- Sem push notifications nativas (adicionado só no iOS 16.4, 2023)
- Sem Background Sync
- Service Worker sem suporte à Periodic Background Sync
- PWA instalada não aparece na App Store (sem discovery)

Distribuição:

- App Store tem ~500M visitas/mês de descoberta orgânica
- PWA depende de URL direta ou search — sem equivalente

Capacidades nativas:

- Bluetooth, NFC, contacts, câmera avançada → ainda limitados ou quebrados em browsers

Fonte honesta: MDN Web Docs + Apple WebKit Bug Tracker. Limitações do iOS são documentadas pela Apple.

PWA vs Nativo — quando escolher

✓ PWA faz sentido

- Usuário base mobile-web
- Conteúdo editorial / e-commerce leve
- Deploy instantâneo sem store review
- Cross-platform com um codebase
- App secundário / fluxo isolado

✗ Nativo faz mais sentido

- Bluetooth, NFC, ARKit/ARCore
- Push em iOS confiável (pré-iOS 16.4)
- App Store discovery é parte do produto
- Gaming ou AR/VR intensivo
- Integração deep com SO (widgets, live activities)

React Native for Web — o que é?

Criado por: Nicolas Gallagher no Twitter/Meta · **Lançamento:** 2018 · necolas.github.io/react-native-web

Proposta: rodar componentes React Native no browser sem reescrever — `View`, `Text`, `TouchableOpacity` viram `div`, `span`, `button` via polyfills DOM.

```
// O mesmo componente funciona em RN (iOS/Android) e na web
import { View, Text } from 'react-native';

export function Card({ title }: { title: string }) {
  return <View><Text>{title}</Text></View>; // web → <div><span>
}
```

Quem usa: Twitter/X usava para compartilhar código entre app e PWA · Shopify com Restyle · Expo Web.

Problema real: componentes pesados do RN (Animated, FlatList) têm overhead no browser. Bundle maior que uma SPA pura.

React Native for Web — quando faz sentido?

Faz sentido quando:

- Já tem app RN e quer versão web **sem segunda codebase**
- Time conhece RN mas não React puro
- Componentes de negócio são simples (formulários, listas, texto)

Não faz sentido quando:

- Experiência web precisa de SEO robusto (SSR difícil)
- App tem UI muito nativa (gestos complexos, animações Reanimated pesadas)
- Projeto novo — prefira React + PWA direto

Posição no mercado (2026): nicho. Expo Web popularizou mas adoção ainda é pequena vs React puro.

Ionic Capacitor — o que é?

Criado por: time da Ionic Framework (Max Lynch, Ben Sperry) · **Lançamento:** 2019 · capacitorjs.com

Proposta: empacotar qualquer web app (PWA, React, Vue, Angular) como app nativo na App Store e Google Play, com acesso a APIs nativas via plugins JS.



plugins JS → câmera · GPS · push · biometria · Bluetooth

Evolução do PhoneGap/Cordova — mesma ideia, mas com TypeScript, suporte a PWA, e integração moderna com

Vite/React.

Ionic Capacitor — quando faz sentido?

Faz sentido quando:

- Já tem PWA ou web app e quer distribuir na store
- Precisa de câmera, push nativo, biometria, Bluetooth — sem abrir mão da web
- Time é 100% web, sem nenhum Swift/Kotlin

Não faz sentido quando:

- App precisa de performance gráfica (jogos, animações 60fps pesadas)
- Integração nativa muito profunda (sensores proprietários, ARKit/ARCore full)

Adoção real (2026): larga — milhares de apps na store usam Capacitor. Empresa de saúde, fintech, B2B SaaS são os nichos mais comuns.

PWA vs RN-Web vs Capacitor

Aspecto	PWA	RN-Web	Capacitor
Distribuição	URL ou install	URL ou install	App Store + Play
Capacidade nativa	Limitada	Limitada	Total via plugins
Tamanho bundle	Pequeno	Médio	Maior
Performance	Web	Web (com RN-style)	Nativo wrapped
Update	Instantâneo	Instantâneo	Via store

Cada um cobre nichos. Não competem diretamente.

PWA dentro do nativo — é possível?

```
// Flutter: WebView abre uma PWA como se fosse nativo
import 'package:webview_flutter/webview_flutter.dart';

WebViewController()
  ..setJavaScriptMode(JavascriptMode.unrestricted)
  ..loadRequest(Uri.parse('https://meupwa.app'));
```

```
// KMP/Android: mesma abordagem
val webView = WebView(context)
webView.settings.javaScriptEnabled = true
webView.loadUrl("https://meupwa.app")
```

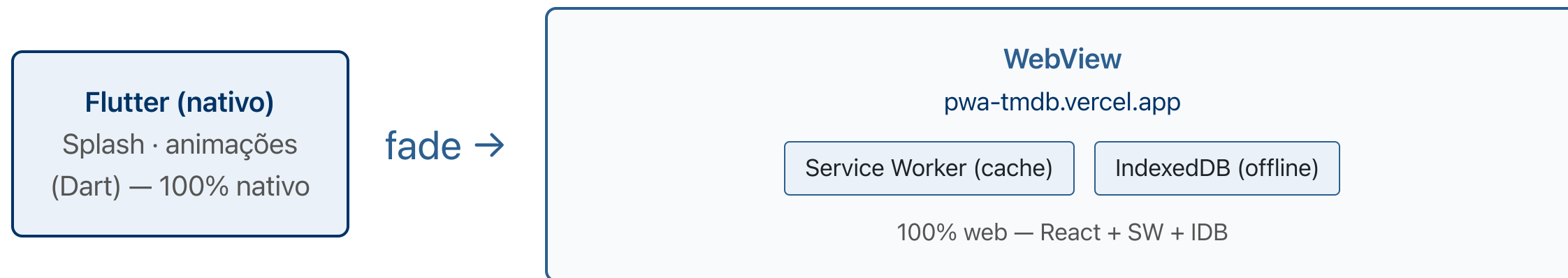
Hybrid shell: app nativo no store + PWA dentro da WebView.

Padrão formalizado pelo **Ionic Capacitor** (capacitorjs.com) — usado em produção por milhares de apps na App Store e Google Play. `webview_flutter` é a implementação Flutter do mesmo conceito.

E se quiser os dois? Flutter + PWA

A escolha **não é binária**. Padrão real usado em produção:

Flutter (ou RN) como shell → WebView carregando o PWA



Usuário não sabe que saiu do Flutter — UA: `CineHub/1.0 FlutterShell`

O User-Agent `CineHub/1.0 FlutterShell` permite ao PWA detectar o contexto e adaptar comportamento.

Ionic Capacitor (capacitorjs.com) popularizou este padrão para devs web. O demo usa a mesma arquitetura via `webview_flutter`.

Atividade 4 — PWA: filmes instalável + offline

Stack: React + Vite + Workbox + IndexedDB (`idb`) + TMDB REST

```
src/services/  
  tmdb.ts          ← ⇄ TODO 1: fetchPopularMovies()  
  db.ts            ← IndexedDB já implementado (saveMovies/loadMovies)  
  
src/repositories/  
  moviesRepository.ts ← NetworkFirst/CacheFirst/SWR já prontos  
  
src/__tests__/unit/  
  02-tmdb-service.test.ts ← ✅ AVALIATIVO — fica vermelho até TODO 1  
  03-useFavorites.test.ts ← ✅ TODO 2: implemente os it.todo()  
  
src/screens/HomeScreen.tsx ← TODO 3: aba Favoritos (3 passos)  
src/hooks/useOfflineStatus.ts ← TODO 3 Passo 3: hook online/offline
```

Entrega: fork → PR `feat(a4-pwa): <login>`

Vite — comandos que você vai usar

Comando	O que faz
<code>npm run dev</code>	Servidor de desenvolvimento (hot reload)
<code>npm run build</code>	Build de produção (gera <code>dist/</code>)
<code>npm run preview</code>	Serve o build de <code>dist/</code> em http://localhost:4173
<code>npm test</code>	Roda testes unitários (Vitest)
<code>npm run test:watch</code>	Vitest em modo watch (re-roda ao salvar)

Para testar o Service Worker offline você **precisa** do `build + preview` — o SW não roda em `dev` .

A4 — Rodando os testes

```
# Na pasta pratica/ (confirme com: ls src/services)
npm test
```

Saída esperada no scaffold (TODO 1 não implementado):

```
FAIL src/__tests__/unit/02-tmdb-service.test.ts
  ✗ 1. retorna lista de filmes via tmdbClient
  ✗ 2. propaga erro quando tmdbClient lança
```

Saída esperada depois do TODO 1:

```
PASS src/__tests__/unit/02-tmdb-service.test.ts
  ✓ 1. retorna lista de filmes via tmdbClient
  ✓ 2. propaga erro quando tmdbClient lança
```

| Teste vermelho = implementar. Verde = entregar.

A4 — Testando offline (demo ao vivo)

Demo rodando: <https://pwa-tmdb-aula4.vercel.app> — abra antes de implementar.

Depois do TODO 1 implementado:

1. `npm run dev` → abre <http://localhost:5173>
2. Deixe carregar os filmes (NetworkFirst busca + salva no IndexedDB)
3. Clique  **Offline** no toggle da tela → app recarrega
4. Os filmes continuam aparecendo (**cache IndexedDB**)
5. Clique  **Limpar cache** → estado vazio (sem rede + sem cache)
6. Clique  **Online** → reload → fetch fresco novamente

Por que o app não quebra offline? `moviesRepository.networkFirst()` tenta a rede, se falha vai pro IndexedDB, se também está vazio retorna `[]` (view mostra estado vazio, não erro).

Pitfalls comuns

- ✗ `skipWaiting()` sem cuidado — clientes abertos podem ficar inconsistentes
- ✗ Cache infinito — esquecer de limpar caches antigos no `activate`
- ✗ `localStorage` ao invés de `IndexedDB` — limite de 5MB e síncrono
- ✗ Service Worker em scope errado — não intercepta o que deveria
- ✗ Não testar em "modo airplane" — só descobre bug em produção
- ✗ Assumir suporte iOS igual ao Chrome — verificar sempre em Safari/iOS

Leituras obrigatórias (pré-Aula 5)

Públicas e gratuitas:

- Archibald, J. — *The Offline Cookbook* · web.dev/offline-cookbook · Google, 2014 (atualizado)
- Workbox docs — *Caching Strategies* · developer.chrome.com/docs/workbox/caching-strategies-overview
- MDN — *Service Worker API* · developer.mozilla.org/pt-BR/docs/Web/API/Service_Worker_API

Hybrid shell (padrão do demo):

- Ionic — *Capacitor: native runtime for web apps* · capacitorjs.com (mesmo padrão do demo; versão para devs web em vez de Flutter)
- `webview_flutter` — pub.dev/packages/webview_flutter (pacote usado no demo)

Casos de uso (dados de indústria — leia criticamente):

- Pinterest PWA case study · web.archive.org — Google Showcase 2017
- AliExpress PWA case study · web.dev/aliexpress
- Twitter Lite Eng Blog 2017 · web.archive.org

Livros (biblioteca PUC / O'Reilly):

- Ater, T. — *Building Progressive Web Apps* (O'Reilly, 2017)

Próxima aula (08/07)

Aula 5 — KMP + BFF, GraphQL e Segurança

Pré-requisito:

- Atividade 4 (PWA) entregue — prazo **08/07** no Canvas
- Capstone CineHub clonado: `compartilhado/capstone/server/` → `npm run seed && npm start`

| KMP (Atividade 5) será feito **ao vivo na Aula 5** — não precisa configurar antes.

Obrigado!

 jackson.96@gmail.com

 linkedin.com/in/3jacksonsmith

 github.com/jacksonsmith

Próxima quarta, 08/07, 19h